

DevOps & Containerization

DevOps unifies **development and operations** to deliver software faster, reliably, and continuously. Containerization technologies like **Docker and Kubernetes** operationalize DevOps at scale by enabling consistent, repeatable deployment environments.

This chapter covers DevOps concepts, Docker fundamentals, container lifecycle, orchestration, and Kubernetes deployment.

1.1 What is DevOps?

DevOps is a **collaborative culture and set of practices** that integrates:

- Software Development (Dev)
- IT Operations (Ops)
- QA & Testing
- Deployment & Monitoring

The objective is to **deliver high-quality software continuously, reliably, and efficiently.**

DevOps focuses on:

- ✓ Collaboration
- ✓ Automation
- ✓ Continuous delivery
- ✓ Fast feedback cycles

It removes silos between teams so that **software moves smoothly from development to production.**

1.2 Goals of DevOps

DevOps aims to:

- **Reduce time-to-market** — deliver features faster
- **Improve deployment frequency** — frequent small releases
- **Increase reliability & stability** — fewer failures and rollbacks
- **Automate repetitive tasks** — reduce manual effort
- **Strengthen feedback loops** — detect and respond to issues quickly
- **Enhance collaboration and ownership** among teams

Overall, DevOps builds **faster, safer, and more predictable software delivery.**

1.3 DevOps Principles

Key guiding principles include:

- **Collaboration & shared ownership**

Development, testing, and operations work as one team.

- **Automation-first approach**

Automate build, test, deployment, monitoring, and infrastructure.

• **Continuous improvement**

Measure, learn, and improve processes continuously.

• **Fail-fast & recover quickly**

Encourage experimentation while ensuring rapid recovery.

• **Infrastructure as Code (IaC)**

Infrastructure is defined and managed through code — consistent and repeatable.

• **Continuous Integration & Continuous Delivery (CI/CD)**

Code is integrated, tested, and deployed continuously.

1.4 DevOps Ecosystem & Phases

The DevOps lifecycle consists of **continuous, repeating stages**:

1. **Plan** — define backlog, user stories, and roadmap
2. **Code** — develop features and perform code reviews
3. **Build** — compile and package source code
4. **Test** — execute automated testing
5. **Release** — manage versioning and approvals
6. **Deploy** — roll out to environments (Dev/Test/Prod)
7. **Operate** — run and manage the application in production
8. **Monitor** — track logs, metrics, and system behavior
9. **Feedback** — feed insights back into planning & improvement

This cycle ensures **continuous delivery and continuous improvement**.

Example DevOps Tools by Phase

Phase	Tools (Examples)
Plan	Jira, Azure Boards
Code	Git, GitHub
Build	Maven, Gradle
Test	JUnit, Selenium
Release	Nexus, Artifactory
Deploy	Docker, Kubernetes
Monitor	Prometheus, Grafana
Automation / CI/CD	Jenkins, GitHub Actions

These tools form the **DevOps toolchain** supporting automation and collaboration across the lifecycle.

2. Containerization (Docker)

2.1 Why Containers?

Traditional software deployment often struggles with:

- **Environment mismatch**

Example: Code works on the developer's laptop but fails on the server due to different OS, library versions, or configurations.

- **Manual configuration effort**

Servers must be configured manually — this is error-prone and inconsistent.

- **Heavy virtual machines (VMs)**

VMs require a full operating system per instance. This consumes large CPU, memory, and disk resources.

- **Scaling difficulty**

Spinning up new VMs takes time and effort.

How Containers Solve These Problems

Containers:

- ✓ Package everything needed to run an application
- ✓ Remove dependency conflicts
- ✓ Are lightweight and fast
- ✓ Allow easy scaling

So the **same container runs identically everywhere** — developer laptop, test server, cloud, etc.

2.2 What is a Container?

A **container** is a lightweight, isolated runtime environment that bundles:

- Application code
- Required libraries
- Runtime dependencies
- Configuration files

This ensures the **application runs the same anywhere**.

Container vs Virtual Machine (VM)

Aspect	Container	Virtual Machine
OS usage	Shares host OS kernel	Has its own OS
Size	Very small (MBs)	Large (GBs)
Startup time	Seconds	Minutes
Performance	Near-native	Slower
Isolation	Process-level	Full OS-level

So, **containers are faster, lighter, and more resource-efficient than VMs**.

2.3 What is Docker?

Docker is the most widely used containerization platform.

Docker allows you to:

- Package an application into a container
- Run that container on any system (Windows, Linux, Mac, Cloud)
- Share and reuse container images
- Maintain consistent environments across teams

In simple terms:

Docker = Standard way to create and run containers.

2.4 Key Docker Components — Explained

Component	Meaning & Role
Docker Engine	The runtime that executes and manages containers. It is the “heart” of Docker.
Docker Image	A blueprint/template for creating containers. Contains everything the app needs.
Docker Container	A running instance of an image . Many containers can run from one image.
Dockerfile	A script that defines how an image is built — step-by-step instructions.
Docker Hub	A public online registry where images are stored and shared.
Docker Compose	A tool used to define and run multiple containers together (e.g., app + database).

Think of it like:

Image = Class

Container = Object

3. Dockerfile & Image Creation

What is a Dockerfile?

A **Dockerfile** is a text file containing instructions to build an image.

Conceptual Example

FROM base-image

COPY source-code

RUN install-dependencies

CMD run-application

Explanation:

Instruction	Meaning
FROM	Specifies the base image (e.g., Ubuntu, Node, Python)
COPY	Copies application files into the image
RUN	Installs required software or libraries
CMD	Command that runs when the container starts

Docker Image Build Process — Steps

1. **Write a Dockerfile**
Define how the image will be built.
2. **Build the Image**
Docker reads the Dockerfile and creates an image.
3. **Image Stored Locally**
Saved in local Docker storage.
4. **Run Containers from Image**
Containers execute using this image.

So the workflow is:

Dockerfile → Image → Container

4. Docker Container Lifecycle

A Docker container goes through multiple stages:

Stage	Description
Create	Container blueprint is defined but not yet running.
Start	Container begins execution. Application runs.
Pause	Container execution is temporarily halted.
Stop	Application is shut down gracefully.
Kill	Container is force-stopped immediately.
Restart	Container stops and starts again.
Destroy (Remove)	Container is permanently deleted.

Note: **Destroying a container does not delete the image.**

5. Essential Docker Management Commands — Explained

Here are key Docker commands and what they do:

Purpose	Command	Meaning
Start Docker service	<code>docker start</code>	Starts an existing container. (Note: starting Docker daemon depends on OS.)
Download an image	<code>docker pull <image></code>	Fetches an image from Docker Hub (e.g., <code>docker pull nginx</code>).
List local images	<code>docker images</code>	Displays all stored Docker images.
Build image	<code>docker build</code>	Creates an image from a Dockerfile.
Run a container	<code>docker run</code>	Starts a new container from an image.
List containers	<code>docker ps -a</code>	Shows running and stopped containers.
Stop container	<code>docker stop</code>	Gracefully stops a running container.
Remove container	<code>docker rm</code>	Deletes a stopped container.
Remove image	<code>docker rmi</code>	Deletes an image from local storage.

Simple Concept Mapping

Container = Running Application

Image = Template

Dockerfile = Recipe

Docker Engine = Kitchen

Docker Hub = Online Image Store



6. Orchestration — Why Kubernetes?

When running **only a few containers**, manual management is possible.

However, in real-world systems **hundreds or thousands of containers** may run across multiple servers.

At that scale, we need:

- **Load Balancing**

Distributes traffic evenly across containers.

- **Auto-Scaling**

Automatically increases or decreases containers based on demand.

- **Self-Healing**

Automatically restarts failed containers and replaces unhealthy ones.

- **Rolling Updates**

Deploys new versions gradually without downtime.

- **Centralized Management**

One control system manages all containers and nodes.

Kubernetes provides all these capabilities — therefore it is used for container orchestration.

7. YAML — Basics

Kubernetes resources are configured using **YAML files**.

YAML is:

- ✓ Human-readable
- ✓ Structured
- ✓ Used to define configuration & desired system state

A Kubernetes YAML file typically contains:

- **metadata** — resource name, labels, identifiers
- **spec** — configuration details such as image, replicas, ports
- **desired state** — how the system should behave

This supports **Infrastructure as Code (IaC)** — meaning infrastructure configurations are stored and version-controlled like software code.

8. Docker Swarm & Stack — Overview

Docker Swarm

Docker Swarm is Docker's **native clustering and orchestration tool**.

It supports:

- Load balancing
- Scaling services
- Service discovery
- High availability



It allows multiple Docker hosts to act as **a single virtual cluster**.

Docker Stack

Docker Stack is used to:

- ✓ Deploy multi-service applications
- ✓ Spread services across Swarm nodes
- ✓ Manage lifecycle of services

Example: Deploying **web app + database + cache together**.

9. Kubernetes — Core Concepts

9.1 What is Kubernetes?

Kubernetes is an **open-source container orchestration platform** used to:

- Deploy applications
- Scale workloads

- Manage lifecycle
- Automatically heal failures

It is designed to run applications **reliably across multiple servers (cluster)**.

9.2 Kubernetes Architecture — Key Components

Component	Description
Cluster	A group of machines (nodes) working together.
Node	A machine (VM or physical) that runs application workloads.
Master Node (Control Plane)	Manages and controls the cluster. Responsible for scheduling, state, and orchestration.
Worker Node	Executes application workloads.
Pod	The smallest deployable unit — usually contains one container (or tightly-coupled containers).
Service	Provides a stable network endpoint to access Pods, even if Pods restart.
Deployment	Manages Pods, replicas, and rolling updates. Ensures required number of Pods are always running.

9.3 Cluster & Service Setup — Conceptual Flow

- 1. Configure Kubernetes Cluster**
Prepare infrastructure and networking.
- 2. Install Control Plane (Master Node)**
Manages the cluster.
- 3. Add Worker Nodes**
Nodes are registered to the cluster.
- 4. Deploy Application as Pods**
Kubernetes runs application containers inside Pods.
- 5. Expose Services**
Applications become reachable.
- 6. Apply Scaling Policies**
Kubernetes auto-scales Pods based on demand.

10. Application Deployment on Kubernetes

Deployment Workflow

- 1. Create Deployment YAML**
Define application image, replicas, config, ports.

2. **Apply Configuration**

Kubernetes reads YAML and sets desired state.

3. **Replicas are Created**

Required number of Pods are launched.

4. **Service Exposes Application**

Provides network access to Pods.

5. **Cluster Balances Traffic**

Traffic is distributed across running Pods.

6. **Auto-Healing Restarts Failed Pods**

Failed Pods are automatically recreated.

This ensures **zero-downtime and resilient deployments**.

11. **Kubernetes Dashboard Usage**

The **Kubernetes Dashboard** is a web-based UI used to:

- Monitor workloads
- View node and resource status
- Manage deployments
- Inspect logs
- Scale replicas

It provides **visual operational control** over the cluster.

12. **DevOps + Containerization — Combined Value**

When DevOps practices are combined with containers and orchestration tools like Kubernetes, organizations gain:

- Faster and frequent deployments
- Immutable & consistent environments
- Easy rollback to previous versions
- Resource-efficient applications
- Automatic scaling
- High availability & reliability
- Reduced production failures

This enables **cloud-native, resilient, modern software delivery**.
